



University of Stuttgart
Institute for Signal Processing and System Theory
Professor Dr.-Ing. B. Yang



Bachelorarbeit S1509

Implementation of an Electromyography Processing Pipeline from Torch model to TinyML on resource-constrained microcontroller platforms

Arbeitstitel, to be defined (TBD)

Author: Andrew Fady Fahmy

Date of work begin: 01/03/2026

Date of submission: 30/06/2026

Supervisor: Jonas Koellner, Prof. Dr. Turker
Ince

Keywords: sEMG, TinyML, Digital Signal
Processing, Deep Learning, re-
gression, Limited-Resource Mi-
crocontroller Platforms

Abstract TBD

Contents

1. Basics and Foundations	1
1.1. EMG and Deep Learning Pipelines for Bioelectric Signal Processing	1
1.1.1. Bioelectric Signals and Human Machine Interaction (HMI)	1
1.1.2. Electromyography (EMG) Basics	1
1.1.3. EMG Signal Preprocessing and Feature Extraction	2
1.1.4. Deep Learning (DL) Pipelines for Bioelectric Signal Processing	2
1.1.5. Deep Learning (DL) for EMG Signal Processing	2
1.2. TinyML and Resource-Constrained Microcontroller Platforms	2
1.2.1. Microcontrollers	2
1.2.2. Limitations and Constraints of Microcontroller Platforms	2
1.2.3. TinyML Overview	2
1.2.4. TinyML Frameworks and tools	4
1.2.5. Applications of TinyML in Bioelectric Signal Processing	4
2. State of The Art (SoTA)	5
2.0.1. EMG Signal Processing Pipelines on Microcontroller Platforms	5
2.0.2. EMG Signal Processing Pipelines with TinyML Frameworks	5
3. Materials and Methods	7
3.1. Dataset Description and Preprocessing	7
3.1.1. NinaPro DB2 Dataset	7
3.1.2. Preprocessing and Feature Extraction	7
3.2. Deep Learning Model Architecture	7
3.3. Hardware	7
3.3.1. ESP32-S3 Specifications	7
3.4. Software and Tools	7
3.4.1. ESP-NN Library	7
3.4.2. LiteRT (Tensorflow Lite) Framework	7
3.4.3. LabStreaming Layer (LSL) for Data Acquisition and Real-Time Processing	7
3.5. Quantization	7
3.5.1. Quantization Types and Schemes	8
3.5.2. Quantization Scheme Used	8
4. Results	9
4.1. Comparison of ML Ops Performance on the ESP32-S3 using Random Weights under different Quantization Schemes	9
4.2. Comparison of different Model Configurations	11

5. Discussion	13
5.1. Software Challenges	13
5.2. LSTM Compatibility and Overhead	13
6. Conclusion	15
6.1. Conclusion	15
6.2. Limitations	15
6.3. Future Work	15
A. Additionally	17
List of Figures	19
List of Tables	21
Bibliography	23

1. Basics and Foundations

1.1. EMG and Deep Learning Pipelines for Bioelectric Signal Processing

1.1.1. Bioelectric Signals and Human Machine Interaction (HMI)

Bioelectric signals are simply defined as a biological signal that originates from a human body, they are the sum of action potentials of cells at an anatomical site such as the heart, brain, or skeletal muscle [1]. These signals have been used extensively in Human Machine Interaction as they provide key insights about human's physiological activity as well as presenting a more natural interaction medium between humans and computers. Although our primary focus in this work is about Electromyography (EMG) signals, we will frequently discuss other commonly researched bioelectric signals such as EEG (Electroencephalogram) and ECG (Electrocardiogram) signals given the abundance of useful HMI research conducted using these 2 domains.

1.1.2. Electromyography (EMG) Basics

Electromyography is the recording of electrical activity in muscle. These electrical impulses originate from the spinal cord and propagate all the way to the peripheral nerves activating muscular contraction [2]. The motor neurons and the muscle fibers they innervate are defined as Motor Units which are recruited by these electrical impulses producing electrical signals. Professional electromyographers analyze characteristics features of these signals (i.e waveforms, firing rates, frequency of abrupt discharges, etc) deriving useful diagnostic information to identify abnormalities and different neural and muscle diseases

The recording of EMG signals is mainly classified into two main categories. The first is intramuscular electromyography (iEMG) which is an invasive technique that includes inserting concentric needle electrodes inserted directly into muscle. The other is surface electromyography (sEMG) which is a non-invasive technique involving the placement of adhesive surface electrodes on skin surface. While iEMG has proven to be more effective in diagnosing medical conditions and abnormalities, sEMG dominate the HMI and gesture recognition sector due to its non-invasive nature, wider accessibility without the need of trained experts and user comfort [3].

In this study, we exclusively focus on sEMG data for the reasons mentioned above

1.1.3. EMG Signal Preprocessing and Feature Extraction

Raw EMG signals suffer from many issues that obscure its insightful value.

1.1.4. Deep Learning (DL) Pipelines for Bioelectric Signal Processing

1.1.5. Deep Learning (DL) for EMG Signal Processing

1.2. TinyML and Resource-Constrained Microcontroller Platforms

1.2.1. Microcontrollers

A Microcontroller unit (MCU) is a self-contained computer implemented on a single integrated circuit. It includes necessary units for regular computation like a fully functioning CPU, integrated memory (RAM), Permanent FLASH memory, IO pins, etc. The adoption of MCUs have seen rapid growth in recent years due to the development of IoT systems (Internet of Things) that needs multiple MCU devices to control, process and compute useful insights from on-ground sensors and communicate it effectively through dedicated channels and networks for further processing and decision-making.

Many key vendors include Arduino (well known for their UNO and Nano modules), Espressif Systems who produce ESP32 MCUs, STMicroelectronics with STM32s, etc. The MCU landscape is constantly shifting every day with new innovations.

1.2.2. Limitations and Constraints of Microcontroller Platforms

Given their cheap cost, small footprint and low-power consumption, MCUs come with many constraints and downgrades relative to personal and heavy computing devices.

For starters, on-chip memory is 3 orders of magnitude smaller than mobile devices and 5-6 orders of magnitude smaller than cloud GPUs [4]. In addition, their CPU clock rate are in MHz range compared to modern personal-use processors which are in the GHz range. Another important aspect is the lack of GPUs or Parallel Processing Units which severely limits the MCU at processing SIMD instructions and vector functions which traditionally limited their use in Graphics and ML fields. Furthermore, Software development for MCUs is notoriously convoluted due to fragmented vendor-specific sdks and frameworks, the wide use of deprecated software in order to accommodate MCU needs and lack of continuous support and maintenance for development frameworks.

1.2.3. TinyML Overview

In the past, the only solution for IoT ML and DL processing was CloudML. This process involved using the communication capabilities of MCUs, e.g WIFI, in order to send raw

Table 1.1.: Comparison of Commercially Available MCUs for Edge ML Applications

MCU	Processor (CPU / cores / clock)	WiFi/BT	Memory & FPU	NN / SIMD	Cost (USD)
ESP32	Xtensa 32-bit LX6, 2 cores, 240 MHz	Yes	SRAM 520 KB PSRAM 8 MB Flash 16 MB ext. FPU SP	ESP-DL / ESP-NN	\$3–8
ESP32-S3	Xtensa 32-bit LX7, 2 cores, 240 MHz	Yes	SRAM 520 KB PSRAM 8 MB Flash 16 MB ext. FPU SP	ESP-DL / ESP-NN	\$4–10
Pi Pico (RP2040)	Arm Cortex-M0+, 2 cores, 133 MHz	W only	SRAM 264 KB PSRAM ext. Flash 2 MB (16 ext.).	CMSIS-NN (no SIMD)	\$4–6
Arduino Nano	ATmega328P (AVR 8-bit), 1 core, 16 MHz	No	SRAM 2 KB; PSRAM no; Flash 32 KB; FPU no	None	\$10–25
STM32 (H7 ex.)	Arm Cortex-M7, 1–2 cores, 480 MHz	No	SRAM 1 MB; PSRAM ext.; Flash 2 MB; FPU DP	CMSIS-NN/DSP (no MVE)	\$8–20
STM32N6	Arm Cortex-M55 + Neural-ART NPU, 1 core, 800 MHz	No	SRAM 4.2 MB; PSRAM ext.; Flash flashless; FPU H/SP/DP	Helium SIMD + NPU 600 GOPS; ST Edge AI	\$5–12

sensor data to cloud servers, with much powerful computing capabilities, for processing and then retrieving commands from those servers in order to take physical actions. While this utilized the full capabilities of Cloud-Cluster and reduced the required specs for IoT devices, this approach suffers from critical issues:

- 1) Reduced output latency due to data + command Round Trips to and from the servers.
- 2) The need for good wifi connections which was not applicable in many cases.
- 3) Exposing security vulnerabilities and privacy concerns.
- 4) The need for developing possibly expensive server infrastructures.

These reasons were the main catalysts for EdgeML research which focussed on optimizing ML frameworks in order to run on mobile edge devices and MCUs. TinyML is a subclass of EdgeML which focusses on squeezing deep learning model on cheap, tiny MCUs through optimizing ML kernels and hardware to fit the constraints at hand.

Since then, TinyML has gained wide traction and adoption in many sectors that previously relied on CloudML. This includes Smart Home devices which benefits from object detection and face recognition DL models, smart watches and wearable devices for instantaneous detection of medical abnormalities and handling emergency situations faster, Human machine interaction specifically developing Gesture recognition software for more immersive experiences, etc.

1.2.4. TinyML Frameworks and tools

Different paradigms have been suggested and developed for TinyML development. The most popular approach for TinyML is through

1.2.5. Applications of TinyML in Bioelectric Signal Processing

2. State of The Art (SoTA)

2.0.1. EMG Signal Processing Pipelines on Microcontroller Platforms

2.0.2. EMG Signal Processing Pipelines with TinyML Frameworks

3. Materials and Methods

3.1. Dataset Description and Preprocessing

3.1.1. NinaPro DB2 Dataset

3.1.2. Preprocessing and Feature Extraction

3.2. Deep Learning Model Architecture

3.3. Hardware

3.3.1. ESP32-S3 Specifications

3.4. Software and Tools

3.4.1. ESP-NN Library

3.4.2. LiteRT (Tensorflow Lite) Framework

3.4.3. LabStreaming Layer (LSL) for Data Acquisition and Real-Time Processing

3.5. Quantization

Normally, Deep learning pipelines rely on floating point operations and embed their parameters (i.e weights and biases) as float64s and float32s. Unfortunately, not all MCU architectures have embedded FPU (floating point units) to handle calculations in floating point notations. In addition, chips that have dedicated FPUs in their architecture, for handling float32 data, suffer from longer processing times and resource intensive operation. Furthermore, floating point numbers necessitate larger spaces in memory for tensor allocations (i.e inputs, outputs, model parameters and activations).

For these reasons, different quantization schemes are unanimously adopted across TinyML literature. It is also the case on many MCU platforms (like the ESP32 and ESP32s3), that optimized low-level kernels for ML ops (like Convolutions and MatMuls) are exclusively written with quantization schemes, like INT8, in mind.

3.5.1. Quantization Types and Schemes

Quantization schemes have been categorized into many section. In this work, we are going to outline schemes that are only relevant to TinyML research.

Categorization according to Data Types

By default, popular frameworks like PyTorch, Tensorflow and JAX utilize single-precision floating point numbers (float32s). Quantization schemes can then be categorized into the output data types that these float32s are converted to.

Float16 quantization uses half the space used by float32s while maintaining the floating point interface as float32s. However, float16 adoption in MCU architectures still remain rare. Most MCUs don't have dedicated kernels for handling float16s and just use float32 interfaces rendering float16 optimizations completely useless. Others are unable to handle float16 data and immediately throw exceptions.

Int8, Int16, Int32, Int64 quantization are by the most used. Most MCU architectures are optimized for handling integer operations which makes int quantization adoption easier and more natural. Low-Level kernels usually use a mix of various integer sizes in their internal Implementation, usually inputs, outputs and weights are int8 while biases use larger formats like int32s.

Categorization according to Data Types

3.5.2. Quantization Scheme Used

4. Results

4.1. Comparison of ML Ops Performance on the ESP32-S3 using Random Weights under different Quantization Schemes

Table 4.1.: LSTM performance across hidden size and layer depth on the ESP32-S3.

Architecture	Quant	Size (KB)	Arena (KB)	Inf (μ s)	MSE	Model Input Shape
LSTM(10, 10, 1)	float32	8.9	2.5	706	0	(1, 1, 10)
LSTM(10, 10, 1)	int8	10.0	3.9	950	0	(1, 1, 10)
LSTM(60, 60, 1)	float32	118.2	5.0	5222	0	(1, 1, 60)
LSTM(60, 60, 1)	int8	42.0	9.4	3716	0	(1, 1, 60)
LSTM(200, 200, 1)	float32	1255.7	12.1	42908	0	(1, 1, 200)
LSTM(200, 200, 1)	int8	339.5	24.8	27738	0	(1, 1, 200)
LSTM(400, 400, 1)	float32	5005.7	0.0	0	–	(1, 1, 400)
LSTM(400, 400, 1)	int8	1295.8	46.9	95646	0	(1, 1, 400)
LSTM(25, 25, 1)	float32	25.3	3.2	1350	0	(1, 1, 25)
LSTM(25, 25, 1)	int8	15.5	5.5	1520	0	(1, 1, 25)
LSTM(25, 25, 2)	float32	52.9	6.3	3229	0	(1, 1, 25)
LSTM(25, 25, 2)	int8	34.3	10.7	3510	0.0104	(1, 1, 25)
LSTM(25, 25, 3)	float32	78.5	8.8	4276	0	(1, 1, 25)
LSTM(25, 25, 3)	int8	50.8	15.5	5082	0.0018	(1, 1, 25)
LSTM(25, 25, 4)	float32	104.0	11.1	5469	0	(1, 1, 25)

Table 4.2.: LSTM performance across window sizes on the ESP32-S3.

Model Input Shape	Architecture	Quant	Size (KB)	Arena (KB)	Model Init Time In Micro	Inf (μ s)	MSE
(1, 1, 25)	LSTM(25, 25, 1)	float32	25.3	3.2	2137	1412	0
(1, 1, 25)	LSTM(25, 25, 1)	int8	15.5	5.5	3428	1531	0
(5, 1, 25)	LSTM(25, 25, 1)	float32	39.4	9.2	4828	3826	0
(5, 1, 25)	LSTM(25, 25, 1)	int8	41.3	21.1	12788	5812	0
(25, 1, 25)	LSTM(25, 25, 1)	float32	110.9	39.0	37316	15341	0
(25, 1, 25)	LSTM(25, 25, 1)	int8	170.7	110.1	104367	31326	0.0007
(100, 1, 25)	LSTM(25, 25, 1)	float32	377.3	148.5	849227	57349	0
(100, 1, 25)	LSTM(25, 25, 1)	int8	653.8	617.1	4469281	184434	0.0014
(300, 1, 25)	LSTM(25, 25, 1)	float32	1090.8	446.7	22476545	181748	0
(300, 1, 25)	LSTM(25, 25, 1)	int8	1946.2	3312.4	78304694	1070880	0.0010

Table 4.3.: CNN performance on the ESP32-S3.

Architecture	Quant	Size (KB)	Arena (KB)	Inf (μ s)	MSE	Model Input Shape	Model Activation
Conv(in=1, out=10, L=1, k=(1, 5))	float32	2.2	7.9	1813	0	(1, 2, 50)	ReLU
Conv(in=1, out=10, L=1, k=(1, 5))	int8	2.5	2.6	1192	0.0003	(1, 2, 50)	ReLU
Conv(in=1, out=10, L=1, k=(1, 5))	float32	2.2	1.7	461	0	(1, 2, 10)	ReLU
Conv(in=1, out=10, L=1, k=(1, 5))	int8	2.5	1.0	540	0.0001	(1, 2, 10)	ReLU
Conv(in=1, out=10, L=1, k=(1, 10))	float32	2.4	26.4	8546	0	(1, 8, 50)	ReLU
Conv(in=1, out=10, L=1, k=(1, 10))	int8	2.6	7.2	3914	0.0003	(1, 8, 50)	ReLU
Conv(in=1, out=10, L=1, k=(1, 10))	float32	2.4	120.1	61808	0	(1, 8, 200)	ReLU
Conv(in=1, out=10, L=1, k=(1, 10))	int8	2.6	30.6	17456	0.0003	(1, 8, 200)	ReLU
Conv(in=1, out=40, L=4, k=(1, 10))	float32	35.0	38.0	138636	0	(1, 8, 100)	ReLU
Conv(in=1, out=40, L=4, k=(1, 10))	int8	15.0	22.5	56215	0.0010	(1, 8, 100)	ReLU
Conv(in=1, out=40, L=4, k=(1, 10))	float32	35.0	65.0	265715	0	(1, 8, 200)	ReLU
Conv(in=1, out=40, L=4, k=(1, 10))	int8	15.0	27.8	106052	0.0014	(1, 8, 200)	ReLU
Conv(in=1, out=10, L=1, k=(10, 10))	float32	5.9	132.1	332771	0	(1, 50, 50)	ReLU
Conv(in=1, out=10, L=1, k=(10, 10))	int8	3.4	33.6	67209	0.0004	(1, 50, 50)	ReLU
Conv(in=1, out=10, L=1, k=(10, 10))	float32	5.9	3612.1	9149353	0	(1, 224, 224)	ReLU
Conv(in=1, out=10, L=1, k=(10, 10))	int8	3.4	903.6	2002723	0.0004	(1, 224, 224)	ReLU
Conv(in=1, out=40, L=4, k=(10, 10))	float32	301.1	47.1	1637829	0	(1, 50, 50)	ReLU
Conv(in=1, out=40, L=4, k=(10, 10))	int8	81.5	134.8	475574	0.0010	(1, 50, 50)	ReLU
Conv(in=1, out=40, L=4, k=(10, 10))	float32	301.1	809.7	25207949	0	(1, 224, 224)	ReLU

Table 4.4.: Dense layers performance on the ESP32-S3.

Architecture	Quant	Size (KB)	Arena (KB)	Inf (μ s)	MSE	Model Input Shape	Model Activation
Dense(50 $\rightarrow$ 2, L=1)	float32	0.5	0.4	49	0	(1, 1, 50)	ReLU
Dense(50 $\rightarrow$ 2, L=1)	int8	0.5	0.4	41	0	(1, 1, 50)	ReLU
Dense(100 $\rightarrow$ 2, L=1)	float32	0.5	0.6	59	0	(1, 1, 100)	ReLU
Dense(100 $\rightarrow$ 2, L=1)	int8	0.5	0.6	55	0	(1, 1, 100)	ReLU
Dense(1280 $\rightarrow$ 2, L=1)	float32	0.5	5.2	507	0	(1, 1, 1280)	ReLU
Dense(1280 $\rightarrow$ 2, L=1)	int8	0.5	5.2	357	0	(1, 1, 1280)	ReLU
Dense(50 $\rightarrow$ 2, L=4)	float32	5.4	0.9	186	0	(1, 1, 50)	ReLU
Dense(50 $\rightarrow$ 2, L=4)	int8	3.2	1.0	274	0.0011	(1, 1, 50)	ReLU
Dense(100 $\rightarrow$ 2, L=4)	float32	13.0	1.1	357	0	(1, 1, 100)	ReLU
Dense(100 $\rightarrow$ 2, L=4)	int8	5.2	1.1	357	0	(1, 1, 100)	ReLU
Dense(1280 $\rightarrow$ 2, L=4)	float32	756.7	6.2	27685	0	(1, 1, 1280)	ReLU
Dense(1280 $\rightarrow$ 2, L=4)	int8	192.7	3.4	14249	0.0005	(1, 1, 1280)	ReLU
Dense(50 $\rightarrow$ 2, L=8)	float32	13.6	1.4	258	0	(1, 1, 50)	ReLU
Dense(50 $\rightarrow$ 2, L=8)	int8	7.5	1.9	370	0	(1, 1, 50)	ReLU
Dense(100 $\rightarrow$ 2, L=8)	float32	36.4	1.7	1632	0	(1, 1, 100)	ReLU
Dense(100 $\rightarrow$ 2, L=8)	int8	13.7	2.4	550	0.0055	(1, 1, 100)	ReLU
Dense(1280 $\rightarrow$ 2, L=8)	float32	3024.8	8.1	106860	0	(1, 1, 1280)	ReLU

4.2. Comparison of different Model Configurations

5. Discussion

5.1. Software Challenges

5.2. LSTM Compatibility and Overhead

6. Conclusion

6.1. Conclusion

6.2. Limitations

6.3. Future Work

A. Additionally

List of Figures

List of Tables

- 1.1. Comparison of Commercially Available MCUs for Edge ML Applications . . 3
- 4.1. LSTM performance across hidden size and layer depth on the ESP32-S3. . . 9
- 4.2. LSTM performance across window sizes on the ESP32-S3. 9
- 4.3. CNN performance on the ESP32-S3. 10
- 4.4. Dense layers performance on the ESP32-S3. 10

Bibliography

- [1] R. Martinek, M. Ladrova, M. Sidikova, R. Jaros, K. Behbehani, R. Kahankova and A. Kawala-Sterniuk, “Advanced Bioelectrical Signal Processing Methods: Past, Present and Future Approach—Part I: Cardiac Signals,” *Sensors*, vol. 21, no. 15, p. 5186, Jul. 2021. [Online]. Available: <https://www.mdpi.com/1424-8220/21/15/5186>
- [2] M. Zheng, M. Crouch and M. Eggleston, “Surface Electromyography as a Natural Human-Machine Interface: A Review,” *IEEE Sensors Journal*, vol. 22, pp. 1–1, May 2022.
- [3] I. V. Valcheva, V. Gandhi and E. Chinellato, “Surface EMG driven gesture recognition using machine learning for robotic applications,” *Discover Robotics*, vol. 2, no. 1, p. 4, Feb. 2026. [Online]. Available: <https://link.springer.com/10.1007/s44430-026-00018-4>
- [4] J. Lin, W.-M. Chen, Y. Lin, J. Cohn, C. Gan and S. Han, “MCUNet: Tiny Deep Learning on IoT Devices,” 2020, version Number: 2. [Online]. Available: <https://arxiv.org/abs/2007.10319>

Declaration

Herewith, I declare that I have developed and written the enclosed thesis entirely by myself and that I have not used sources or means except those declared.

This thesis has not been submitted to any other authority to achieve an academic grading and has not been published elsewhere.

Stuttgart, TBD Date of sign.

 Andrew Fady Fahmy